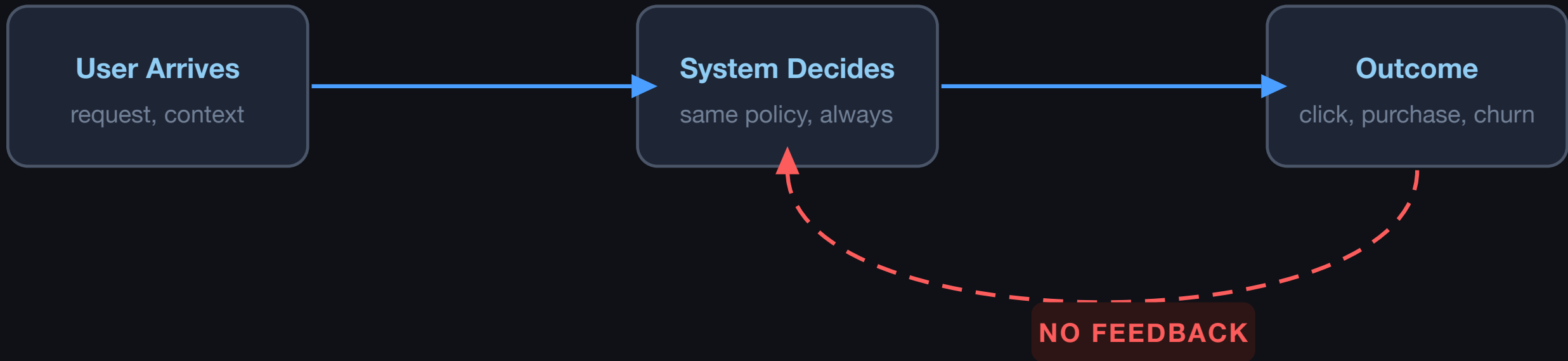


BanditDB

**In-memory decision database
that learns from feedback**

Every Served Request Is a Decision

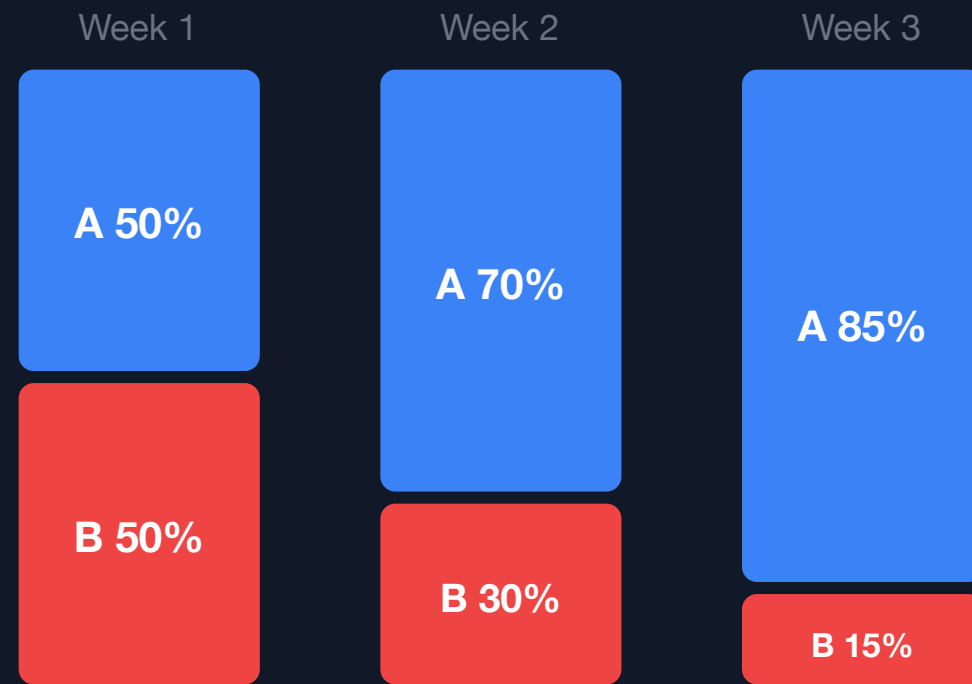


A/B TEST



Stuck — no learning, no personalisation

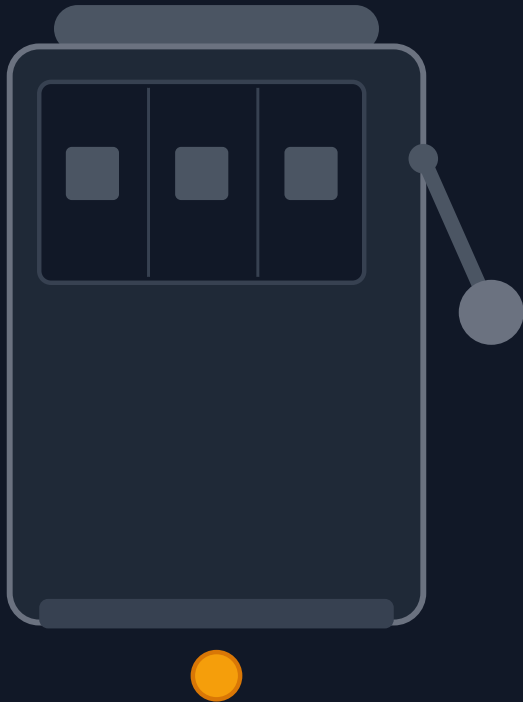
CONTEXTUAL BANDIT



Learns · best strategy earns more traffic

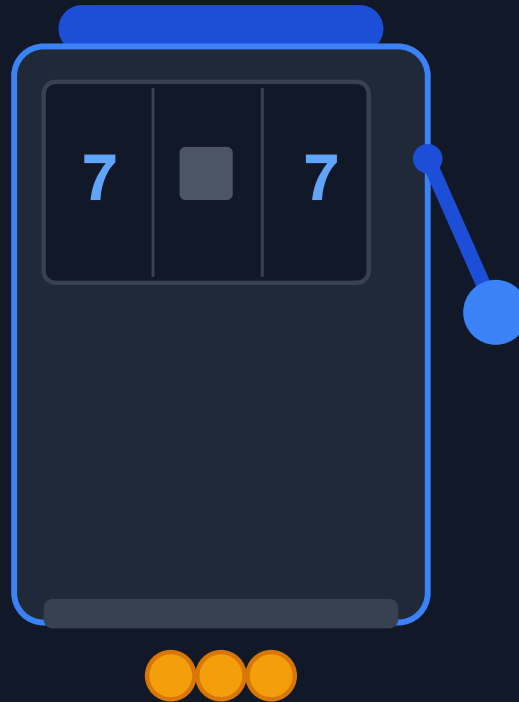
A/B tests pay the full cost of exploration — and throw away context signal entirely.

The One-Armed Bandit



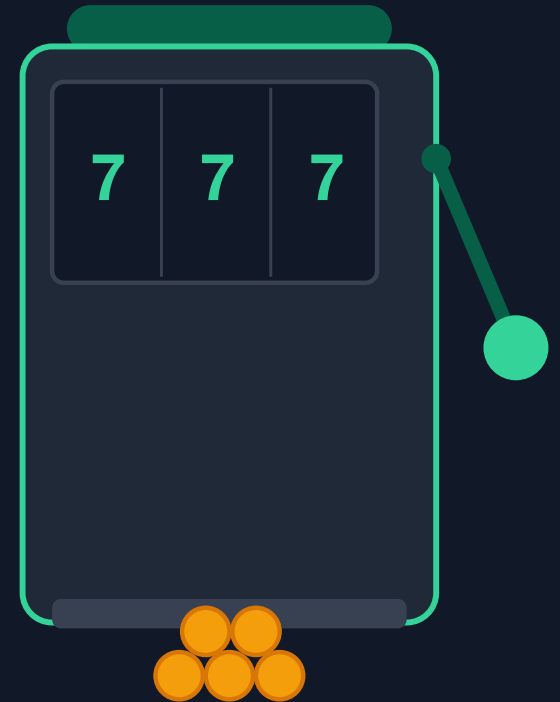
~20%

Explore less



~50%

Keep exploring



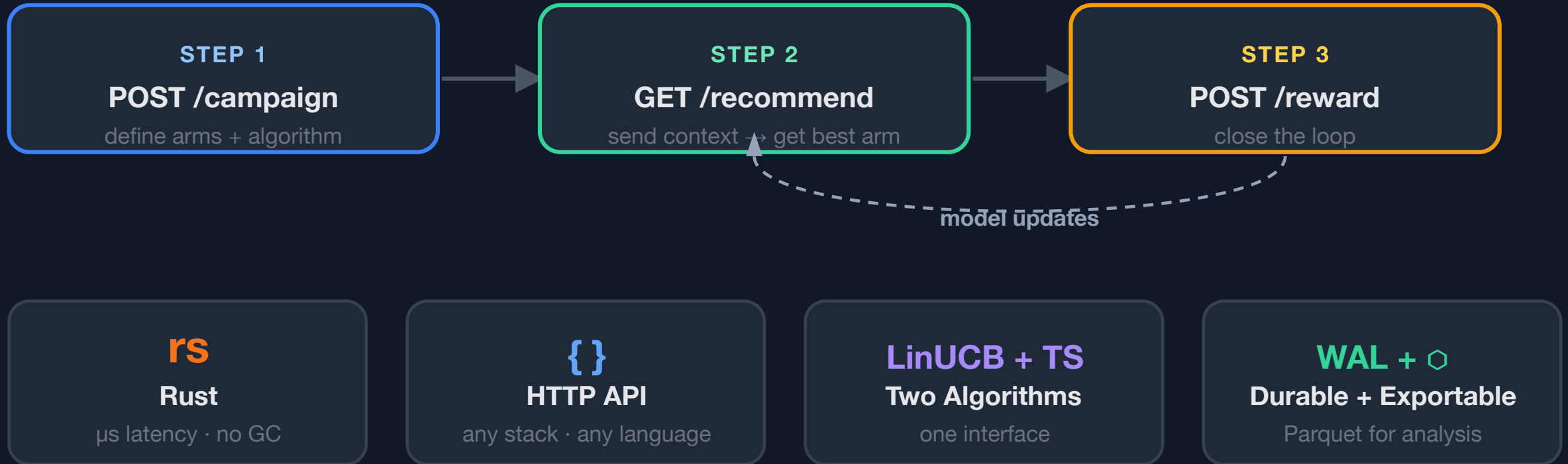
~80%

Exploit more

Each machine hides its payout. You only learn by pulling. Which lever do you pull next?

BanditDB

A contextual bandit engine built for production



Three API calls. Two algorithms. One engine. Ship in an afternoon.

$$E[r_{a,t} | x_{a,t}] = x_{a,t}^T \theta_a^*$$



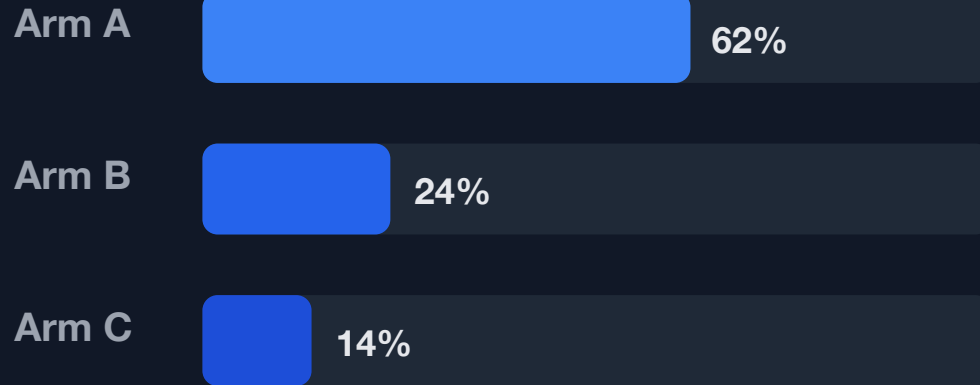
LIVE DEMO

EXAMPLE

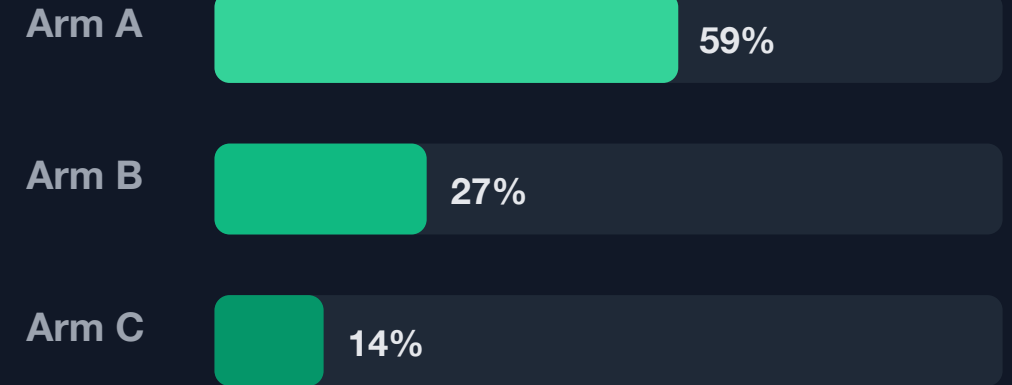
Sleep campaign · 3 arms · dim 5 · sandbox.banditdb.com

CAUSAL FOREST ANALYSIS

BANDIT DISTRIBUTION



CAUSAL ASSIGNMENT



≈

Distributions match — model has converged to the correct causal structure ✓

ATE per arm

Did each arm causally drive reward?
Average Treatment Effect · 90% CI

Feature Importance

Which context dims drive arm selection?
Heterogeneous treatment effects · CATE

Winning Segments

Which user profiles belong to each arm?
High / low feature splits · per arm

Regret tells you the bandit is working. Causal Forest tells you it learned the right thing.

SETUP & CONFIGURATION

`docker-compose.yml`

```
services:
banditdb:
  image:      simeonlukov/banditdb:latest
  ports:     ["8080:8080"]
  volumes:    banditdb_data:/data
  environment:
```

```
DATA_DIR      : /data
```

```
BANDITDB_API_KEY      : your-secret-key
```

```
BANDITDB_REWARD_TTL_SECS      : 86400
```

```
BANDITDB_CHECKPOINT_INTERVAL  : 10000
```

```
BANDITDB_MAX_WAL_SIZE_MB      : 50
```

```
$ docker compose up -d
```

DATA_DIR

default: "."

WAL, checkpoints and Parquet exports location

BANDITDB_API_KEY

unset = open

Enables auth. Unset = open dev mode (no header needed)

BANDITDB_REWARD_TTL_SECS

default: 86400

How long to hold context while waiting for its reward

BANDITDB_CHECKPOINT_INTERVAL

disabled

Auto-checkpoint after every N rewarded events

BANDITDB_MAX_WAL_SIZE_MB

disabled

Auto-checkpoint when WAL exceeds N MB

All env vars are optional with safe defaults. Zero config needed to get started.

METADATA — YOUR SCHEMA LIVES HERE

TRADITIONAL APPROACH

```
-- campaigns table
CREATE TABLE campaigns (
  id VARCHAR,
  owner VARCHAR,
  feat_names TEXT[],
  version VARCHAR,
  description TEXT,
  ...
);

-- Need a migration every time
-- requirements change
```

✗ schema migrations forever

✓ Free-form — any structure

✓ Persisted through checkpoints

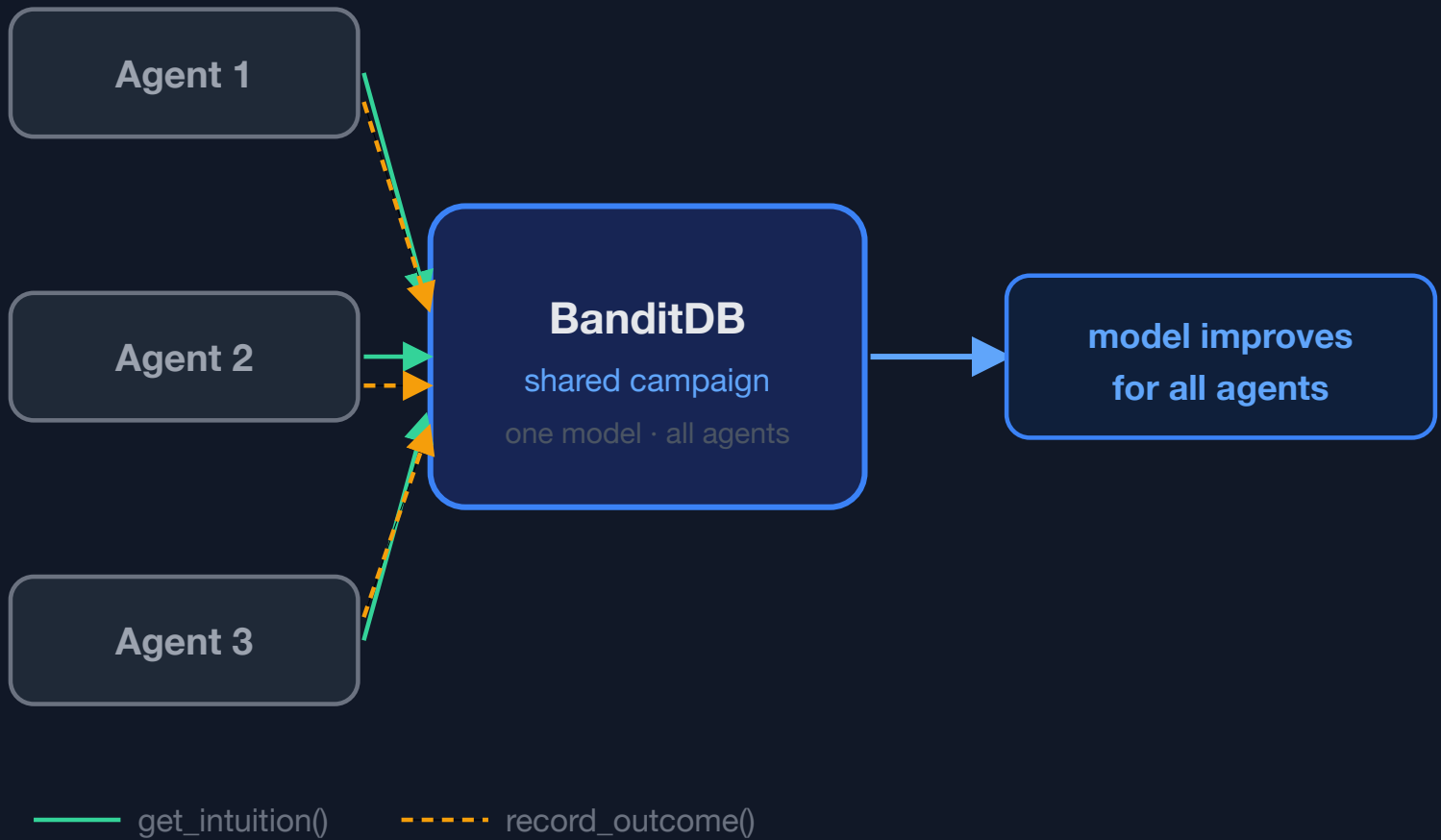
BANDITDB APPROACH

```
{
  "campaign_id":    "sleep",
  "arms": [...],
  "feature_dim":    5,
  "metadata": {
    "owner":        "wellness-team",
    "features":     ["sex", "age", ...],
    "version":      "1.2",
    "anything":     "you need"
  }
}
```

✓ any JSON · no migrations ever

✓ Returned by the API

MCP & AGENTIC USE



```
$ pip install banditdb-python
```

MCP TOOLS

```
get_intuition (campaign, context)
```

```
record_outcome (iid, reward)
```

```
create_campaign (...)
```

```
list_campaigns ()
```

```
campaign_diagnostics (id)
```

prompt strategy · negotiation · call center routing

Agents share one campaign — every interaction improves the model for the entire swarm.